

IPN

Instituto Politécnico Nacional



UPIIZ

Unidad Académica Interdisciplinaria de Ingeniería Campus Zacatecas

Unidad de Aprendizaje:

Paradigmas de Programación.

Profesor:

Efraín Arredondo Morales.

Ejercicio 15:

Sistema de Figuras Geométricas utilizando Clases Abstractas

Alumno:

Cristopher Farid Vázquez Sánchez

2026670199

Semestre y Grupo:

3IM1.

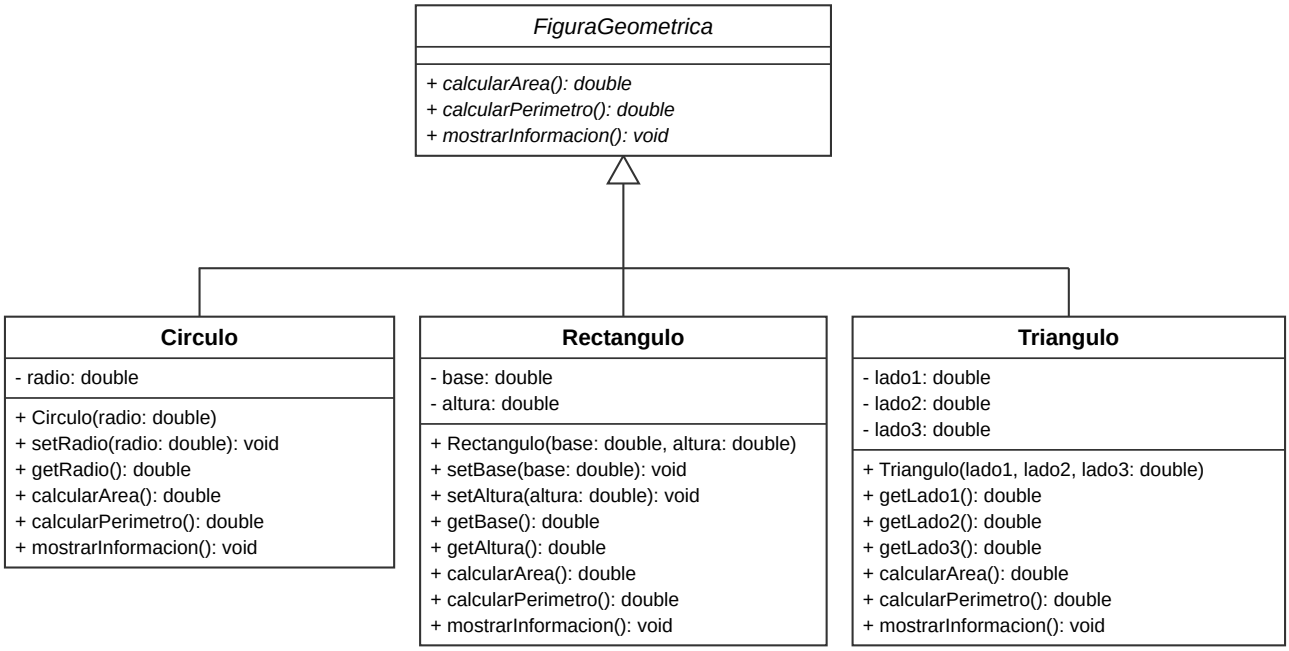
Fecha de Entrega:

29/10/2026

Repositorio:

https://github.com/crissanzzz07/EJE15_CFVS

DIAGRAMA DE CLASES



Clase FiguraGeometrica

```java
package org.upiiz.ipn;

//Clase abstracta: define lo que TODAS las figuras deben saber hacer
public abstract class FiguraGeometrica {

    //Metodos abstractos (sin cuerpo): cada figura concreta debe implementarlos
    public abstract double calcularArea();

    public abstract double calcularPerimetro();

    public abstract void mostrarInformacion();
}
```

Clase Circulo

```java
package org.upiiz.ipn;

//Clase hija o sub clase
public class Circulo extends FiguraGeometrica {
    private double radio;

    public Circulo(double radio) {
        setRadio(radio);
    }

    public void setRadio(double radio) {
        //Validacion: no se permiten valores menores o iguales a cero
        if (radio <= 0) {
            throw new IllegalArgumentException("El radio debe ser mayor que cero");
        }
        this.radio = radio;
    }

    public double getRadio() {
        return radio;
    }

    @Override
    public double calcularArea() {
        return Math.PI * radio * radio;
    }

    @Override
    public double calcularPerimetro() {
        return 2 * Math.PI * radio;
    }

    @Override
    public void mostrarInformacion() {
        System.out.println("Figura: Círculo");
        System.out.println("Radio: " + radio);
        System.out.printf("Área: %.2f%n", calcularArea());
        System.out.printf("Perímetro: %.2f%n", calcularPerimetro());
    }
}
```

Clase Rectangulo

```java
package org.upiiz.ipn;

//Clase hija o sub clase
public class Rectangulo extends FiguraGeometrica {
    private double base;
    private double altura;

    public Rectangulo(double base, double altura) {
        setBase(base);
        setAltura(altura);
    }

    public void setBase(double base) {
        //Validacion: no se permiten valores menores o iguales a cero
        if (base <= 0) {
            throw new IllegalArgumentException("La base debe ser mayor que cero");
        }
        this.base = base;
    }

    public void setAltura(double altura) {
        if (altura <= 0) {
            throw new IllegalArgumentException("La altura debe ser mayor que cero");
        }
        this.altura = altura;
    }

    public double getBase() {
        return base;
    }

    public double getAltura() {
        return altura;
    }

    @Override
    public double calcularArea() {
        return base * altura;
    }

    @Override
    public double calcularPerimetro() {
        return 2 * (base + altura);
    }

    @Override
    public void mostrarInformacion() {
        System.out.println("Figura: Rectángulo");
        System.out.println("Base: " + base);
        System.out.println("Altura: " + altura);
        System.out.printf("Área: %.2f%n", calcularArea());
        System.out.printf("Perímetro: %.2f%n", calcularPerimetro());
    }
}
```

Clase Triangulo

```java
package org.upiiz.ipn;

//Clase hija o sub clase
public class Triangulo extends FiguraGeometrica {
    private double lado1;
    private double lado2;
    private double lado3;

    public Triangulo(double lado1, double lado2, double lado3) {
        //Validacion: no se permiten valores menores o iguales a cero
        if (lado1 <= 0 || lado2 <= 0 || lado3 <= 0) {
            throw new IllegalArgumentException("Los lados deben ser mayores que cero");
        }
        //Desigualdad triangular: la suma de dos lados debe ser mayor que el tercero
        if (lado1 + lado2 <= lado3 || lado1 + lado3 <= lado2 || lado2 + lado3 <= lado1) {
            throw new IllegalArgumentException("Los lados no cumplen la desigualdad triangular");
        }
        this.lado1 = lado1;
        this.lado2 = lado2;
        this.lado3 = lado3;
    }

    public double getLado1() {
        return lado1;
    }

    public double getLado2() {
        return lado2;
    }

    public double getLado3() {
        return lado3;
    }

    @Override
    public double calcularArea() {
        //Formula de Heron
        double s = calcularPerimetro() / 2;
        return Math.sqrt(s * (s - lado1) * (s - lado2) * (s - lado3));
    }

    @Override
    public double calcularPerimetro() {
        return lado1 + lado2 + lado3;
    }

    @Override
    public void mostrarInformacion() {
        System.out.println("Figura: Triángulo");
        System.out.println("Lado 1: " + lado1);
        System.out.println("Lado 2: " + lado2);
        System.out.println("Lado 3: " + lado3);
        System.out.printf("Área: %.2f%n", calcularArea());
        System.out.printf("Perímetro: %.2f%n", calcularPerimetro());
    }
}
```

Main

```java
package org.upiiz.ipn;

import java.util.ArrayList;
import java.util.List;

//Repositorio: https://github.com/crissanzzz07/EJE15_CFVS
public class Main {
    public static void main(String[] args) {
        //Lista de tipo FiguraGeometrica: guarda cualquier figura (polimorfismo)
        List<FiguraGeometrica> figuras = new ArrayList<>();
        figuras.add(new Circulo(5));
        figuras.add(new Rectangulo(4, 6));
        figuras.add(new Triangulo(3, 4, 5));

        System.out.println("=== SISTEMA DE FIGURAS GEOMÉTRICAS ===");
        System.out.println();

        //Se llama desde la referencia de la clase base; cada figura responde a su manera
        for (FiguraGeometrica figura : figuras) {
            figura.mostrarInformacion();
            System.out.println();
        }
    }
}
```

Consola

```
=== SISTEMA DE FIGURAS GEOMÉTRICAS ===

Figura: Círculo
Radio: 5.0
Área: 78.54
Perímetro: 31.42

Figura: Rectángulo
Base: 4.0
Altura: 6.0
Área: 24.00
Perímetro: 20.00

Figura: Triángulo
Lado 1: 3.0
Lado 2: 4.0
Lado 3: 5.0
Área: 6.00
Perímetro: 12.00
```

- *Explicación breve de cómo se aplicaron la abstracción, la herencia y el polimorfismo*

La abstracción se aplicó con la clase abstracta FiguraGeometrica, que declara tres métodos abstractos (calcularArea(), calcularPerimetro() y mostrarInformacion()) sin cuerpo. Como es abstracta no se pueden crear objetos de ella directamente: funciona solo como plantilla que define qué debe saber hacer cualquier figura. Su nombre se escribe en cursiva en el diagrama de clases, que es como UML indica que una clase es abstracta.

La herencia se aplicó con extends: Circulo, Rectangulo y Triangulo heredan de FiguraGeometrica y están obligadas a implementar los tres métodos con @Override, cada una con su propia fórmula. En Triangulo el área se calcula con la fórmula de Herón a partir del semiperímetro. Además, los constructores validan que las dimensiones sean mayores que cero y Triangulo verifica la desigualdad triangular, lanzando una IllegalArgumentException si no se cumple.

El polimorfismo se ve en la clase Main: los objetos se guardan en una List<FiguraGeometrica> y se recorren con un ciclo que llama a mostrarInformacion() desde la referencia de la clase base. Java ejecuta la versión de cada figura y Main no realiza ningún cálculo geométrico. La composición y la agregación no se utilizaron porque ninguna clase contiene objetos de otra como atributo; la única relación entre las clases es la herencia.